

Highlights

From Potential to Confirmed: Effective Detection of Atomicity Violations in Interrupt-Driven Programs via Guided Memory Access Graph

Zixuan Yuan, Bin Yu, Xincheng Wang, Xu Lu, Cheng Wen, Wensheng Wang, Hao Wang, Chu Chen, Cong Tian

- A novel BMC approach for atomicity violations in interrupt-driven programs.
- Extracts key partial orders via symbolic encoding of memory events.
- Constructs partial order-guided memory access graphs on-the-fly.
- BMC4AV improves precision while reducing time and space overhead.

From Potential to Confirmed: Effective Detection of Atomicity Violations in Interrupt-Driven Programs via Guided Memory Access Graph

Zixuan Yuan^a, Bin Yu^a, Xincheng Wang^a, Xu Lu^a, Cheng Wen^a, Wensheng Wang^a, Hao Wang^b, Chu Chen^c, Cong Tian^a

^a*Xidian University, Xi'an, 710126, Shaanxi, China*

^b*Taiyuan University of Technology, Taiyuan, 030024, Shanxi, China*

^c*Qufu Normal University, Rizhao, 276826, Shandong, China*

Abstract

Interrupt-driven programs are commonly used in safety-critical domains. However, non-deterministic executions of interrupt service routines can introduce concurrency defects. As one serious issue, an atomicity violation arises when a sequence of operations expected to execute atomically is unexpectedly interrupted. The random and asymmetric preemption makes it still a challenge to detect such defects precisely and efficiently. To address this, we propose a bounded model checking approach based on a partial order-guided memory access graph. Specifically, all potential atomicity violations are first identified based on the symbolic encoding of memory events, so that the corresponding key partial orders can be extracted. With the guidance of these partial orders, one customized memory access graph is further finely constructed on-the-fly, enabling the effective confirmation of real atomicity violations. The proposed approach has been implemented as a bounded model checker named BMC4AV, and evaluated on academic benchmarks and real-world embedded programs. Experimental results demonstrate that BMC4AV achieves higher precision while reducing time and space overhead compared to state-of-the-art tools.

Keywords:

Interrupt-driven Program, Atomicity Violation, Concurrency Defect, Memory Access Graph, Model Checking

1. Introduction

Interrupts are prevalent in almost all computing systems, including safety-critical embedded software and high-end information systems [1, 2, 3]. This type of system heavily relies on Interrupt Service Routines (ISRs) to interact with hardware and promptly respond to external stimuli. Although this architecture enhances system responsiveness and resource utilization, the non-deterministic nature of interrupts poses significant challenges for software development [4, 5].

Specifically, the primary difficulty in developing interrupt-driven programs arises from the unpredictable triggering of interrupts and their asymmetric preemption. At any moment, an ISR may preempt an ongoing task execution, potentially modifying shared data [6, 7, 8, 9, 10]. If the access to shared data is not properly synchronized or protected, concurrency errors may be introduced, resulting in severe safety issues such as logical inconsistencies or system crashes.

Among these concurrency errors, atomicity violations represent one of the most common and critical categories [11, 12]. Concretely, an atomicity violation occurs when a sequence of operations that is expected to execute without interference is interrupted by an ISR that modifies shared data. Although several approaches have been proposed to detect atomicity violations in multi-threaded programs [13, 14, 15, 16, 11, 17, 18, 19, 20, 21, 1, 22], they cannot be directly applied to interrupt-driven programs due to differences in synchronization and preemption mechanisms [23].

Existing approaches to detecting atomicity violations in interrupt-driven programs mainly rely on static analysis or model checking. For static analysis, as a representative tool, intAtom [23] utilizes the data-flow analysis to check the feasibility of paths between consecutive accesses in each preempted task. However, due to the lack of precise reachability analysis, it cannot handle the complexity caused by non-deterministic interleavings among tasks with different priorities. For model checking, CPA4AV [24] uses the flow-sensitive analysis with abstract reachability trees to identify potential violations, but it struggles to handle complex data types. More recently, NIChecker [25] transforms an interrupt-driven program into a sequential one through lazy sequentialization, and then applies CBMC [26] to detect violations. Yet, NIChecker requires users to manually specify violation types, preventing full automation. In addition, its assertion-insertion strategy may cause false negatives due to semantic mismatches, and the original CBMC

encoding is not optimized, leading to low efficiency.

In this paper, we propose a bounded model checking approach to detect atomicity violations in interrupt-driven programs. The core innovation is a precise and efficient detection strategy based on a partial order-guided Memory Access Graph (MAG). Specifically, the targeted program is first symbolically encoded, and then partial orders are constructed to capture possible interleavings among tasks with different priorities. From these, all potential violations are identified and the corresponding key partial orders are extracted to further guide the subsequent detection process. In the back-end, one customized MAG is finely maintained within the solver to represent the concrete program execution. With iterative guidance by the key partial orders from the front-end, a stable and acyclic graph is incrementally constructed, ultimately confirming the presence of real violations. This design improves precision by leveraging key partial orders to detect atomicity violations and enhances efficiency by pruning unnecessary constraints through the MAG.

We have implemented our approach as a tool named BMC4AV, built on the open-source model checker CBMC [26] and the SAT solver MiniSat [27]. BMC4AV is evaluated on 25 academic benchmarks and 18 real-world embedded programs, with performance compared against state-of-the-art (SOTA) tools. Results on the academic benchmarks show that BMC4AV achieves the highest precision among all competitors. Additionally, on the 18 real-world programs, BMC4AV outperforms the latest detector NICKChecker, reducing detection time by 92.0% and exploration space by 62.0%, while also uncovering 57 false negatives missed by NICKChecker. Ablation studies further confirm that the partial order guidance strategy improves the detection precision without sacrificing efficiency, and the MAG-based detection strategy significantly reduces both the state space and detection time.

The contributions of this paper are summarized as follows:

- (1) A bounded model checking approach is proposed for detecting atomicity violations in interrupt-driven programs, introducing a partial order-guided MAG that leverages key partial orders as guiding constraints and incrementally confirms real violations.
- (2) A practical detection tool named BMC4AV is developed for interrupt-driven C programs to implement the proposed approach.
- (3) Comprehensive experiments are conducted and the results show that BMC4AV achieves higher precision while reducing time and space overhead compared to state-of-the-art tools.

The remainder of this paper is structured as follows. Section 2 introduces the preliminaries and motivation. Section 3 presents the framework of our approach, while the detailed modules are elaborated on Section 4 and Section 5. Section 6 details the implementation and evaluation, and Section 7 discusses the related work. Finally, the conclusion is presented in Section 8.

2. Background and Motivating Example

2.1. Interrupt-driven Programs

Interrupts are widely used in real-time embedded systems to handle events requiring immediate attention. When an interrupt occurs, the processor suspends the current execution, saves its state, and invokes the corresponding ISR [28, 29], after which normal execution resumes. A program is termed interrupt-driven when its most critical functions are managed by interrupts.

Characterized by a concurrency model, an interrupt-driven program can be formally defined as $P = Main \parallel ISR$, where $Main$ is the main task and $ISR = isr_1 \parallel \dots \parallel isr_n$ represents ISRs which handle external inputs or periodic tasks. For simplicity, $Main$ and all ISRs in a program P are called tasks, denoted by $Task(P) = \{Main, isr_1, \dots, isr_n\}$. Furthermore, the symbol $Pri(isr_n)$ is used to denote the priority of isr_n , where a higher numerical value indicates a higher priority, and $Pri(Main)$ is defined as 0 representing the lowest priority.

Notably, the concurrency scheduling mechanism used in interrupt-driven programs is different from that in thread-based ones. Specifically, in interrupt-driven programs, a priority-based asymmetric preemption model is employed. This means that only higher-priority ISRs can preempt lower-priority ones, and ISRs can also preempt the main task. Moreover, unlike multi-threaded programs, which can manage the execution order through blocking mechanisms, one ISR in interrupt-driven programs generally does not block once its execution begins, unless it is preempted by a higher-priority ISR. Along with it comes the increased complexity of system behavior, making the program analysis more challenging.

2.2. Atomicity Violation

Atomicity is an important property to ensure the semantic correctness of an interrupt-driven program. Also known as serializability, it means the

execution effect of concurrent operations is equivalent to their serial execution. When operations expected to be atomic are interrupted by other flows, atomicity violations occur, leading to unintended behavior.

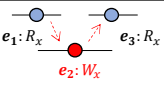
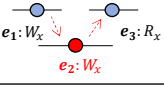
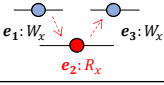
Pattern	Description	Illustration	Problem
(R, W, R)	A write operation is inserted between two consecutive read operations.		The interruption causes the values of the two read operations to be discrepant, breaking data inconsistent.
(W, W, R)	A read operation is performed after two consecutive write operations.		The local read operation expects to read the value written locally, but it reads the value written by interruption.
(W, R, W)	A read operation interrupts two consecutive write operations.		The data read during the interruption is an intermediate result between two local write operations.

Figure 1: Three atomicity violation patterns

The concept of serializability [11] is adopted to characterize access interleaving patterns that correspond to atomicity violations in interrupt-driven programs. Specifically, for an interleaving pattern represented as a triplet (e_1, e_2, e_3) , where e_1 , e_2 and e_3 are read or write operations, it is assumed that e_1 and e_3 are consecutive operations in a lower-priority task, while e_2 belongs to a higher-priority task. The three unserializable interleaving patterns can be summarized as Fig. 1, where the set of these three types of unserializable interleaving patterns is referred to as **Patterns**.

It should be noted that, since empirical studies on interrupt-driven programs [23, 30] have shown that two consecutive accesses to a shared variable often occur within the same function, e_1 and e_3 are also restricted to the same function in this paper. However, unlike previous works [23, 24, 25, 31] considering four types of **Patterns**, we only focus exclusively on three types, ensuring a more precise and targeted analysis. Our decision to exclude **Pattern: (R, W, W)** is based on an in-depth analysis of Racebench 2.1¹, which is the most widely used open-source academic benchmark for evaluating atomicity violation detectors, as well as real-world programs detected in our experiment. The analysis reveals that this pattern typically falls into two categories in practice:

- **Impact on control flow:** Consider the example code in Fig. 2(a), extracted from `svp_simple_027` in Racebench 2.1. In this scenario, the write

¹<https://github.com/chenruihua/racebench>

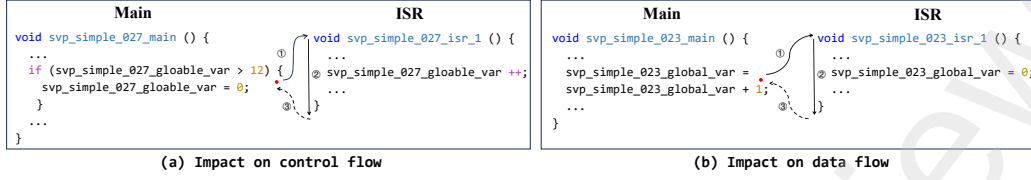


Figure 2: Two cases of Pattern: (R,W,W) from Racebench 2.1

operation `svp_simple_027_gloable_var = 0` in the main task is guarded by the read operation `svp_simple_027_gloable_var > 12`. Pattern: (R,W,W) assumes that the high-priority ISR may preempt between the above operations and execute the write operation `svp_simple_027_gloable_var ++`. However, in practice, the read operation in the main task has already been completed before the write in the ISR occurs. Therefore, the evaluation of the branch condition is unaffected, and the control flow leading to the write inside the branch remains unchanged.

- **Impact on data flow:** Consider the example code in Fig. 2(b), extracted from `svp_simple_023` in Racebench 2.1. In this scenario, the main task computes `svp_simple_023_gloable_var = svp_simple_023_gloable_var + 1`, where the write depends on the value previously read from the same variable. In practice, once the read operation has been executed, the read value is already stored as a temporary variable in the current task context. Therefore, when the ISR preempts and executes `svp_simple_023_gloable_var = 0` to update the same memory location, this write does not change the value already read by the main task. Consequently, the computation in the main task is unaffected, and the data dependence between the read and write operations is preserved.

Based on the above analysis, we treat Pattern: (R,W,W) as benign and exclude it from detection. However, it should be pointed that, our approach can be directly applied to this pattern if needed.

2.3. Motivating Example

The example program `Exa.c` in Fig. 3(a) illustrates atomicity violations in interrupt-driven programs. In the following sections, this example also demonstrates our approach. Since the only ISR `isr` is disabled by default at the beginning, these assignment operations in the main task are not affected by ISR preemption. Afterwards, `isr` is enabled in Line 4 of the main task,

allowing preemption to occur. For shared variable x , the write operation in Line 3 and the subsequent read operation in Line 5 are considered atomic, and these two values are expected to be the same. However, when the main task is preempted by *isr*, the value of x is rewritten, violating the expected atomicity.

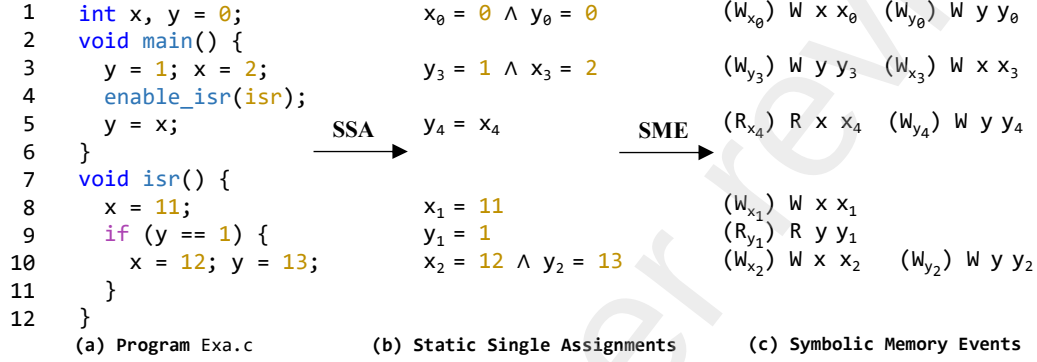


Figure 3: Program Exa.c and corresponding SSA and SME

It should be emphasized that, due to the non-blocking nature of interrupt-driven programs, the write operation on the shared variable x in Line 8 of *isr* is overwritten by the write operation in Line 10 on the condition that $y == 1$ is satisfied. This means that the read operation on x in Line 5 is overwritten by the write operation in Line 10, rather than by the write operation in Line 8. Therefore, the three accesses to x in Lines 3, 10, and 5 form the specific Pattern: (W,W,R). From this example, the following two observations can be drawn to detect atomicity violations effectively:

- Obs. 1: Determining the actual read–write relations of shared variables is vital for detection precision. For example, in program Exa.c, the read of x at Line 5 cannot observe the write at Line 8, as it is overwritten by a later write at Line 10. Hence, the interleaving of operations at Lines 3, 8, and 5, though seemingly plausible, does not constitute a real violation. Accurately identifying which writes are observed by reads is therefore essential to distinguish real violations from spurious ones. To this end, we propose a partial order–guided strategy that validates read sources and eliminates spurious atomicity violations.
- Obs. 2: Avoiding redundant order constraints is critical for detection efficiency. In interrupt-driven programs, complete constraint encoding is

typically required due to non-deterministic and asymmetric preemptions. For example, in program **Exa.c**, prior work encodes the constraint that the write to y at Line 5 precedes the write at Line 10. This constraint, however, is unnecessary for detecting violations, since enforcing it implies a preemption after Line 5 where no violation can occur. Such redundant constraints substantially increase formula complexity and lead to time and space overhead. To address this, we propose a MAG-based strategy that infers only the essential orders on-the-fly, leveraging the implicit correlations within partial orders.

3. Framework of Our Approach

In this section, a framework is proposed for statically detecting atomicity violations in interrupt-driven programs. As shown in Fig.4, it consists of two phases: *Symbolic Encoding (SE)-based Potential Violations Identification* and *Memory Access Graph (MAG)-based Real Violations Confirmation*.

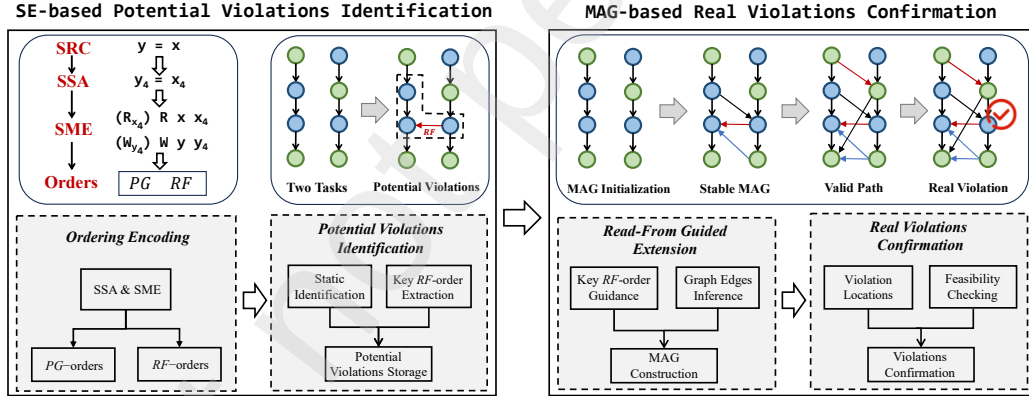


Figure 4: Framework of atomicity violations detection

Specifically, in the *SE-based Potential Violations Identification* phase as the front-end (Sec. 4), the source code is first parsed as the input and transformed into Static Single Assignments (SSAs) and Symbolic Memory Events (SMEs) within the *Ordering Encoding* module. With this basis, the ordering information of memory events can be obtained and corresponding partial orders are established. Subsequently, the *Potential Violations Identification* module identifies all potential atomicity violations and extracts

corresponding key read-from orders. Further, in the *MAG-based Real Violations Confirmation* phase as the back-end (Sec. 5), the *Read-From Guided Extension* module leverages key read-from orders and applies our proposed edge inference rules to iteratively construct a stable and acyclic MAG. Finally, the *Real Violations Confirmation* module locates potential violations using key read-from orders and confirms real atomicity violations on-the-fly.

It should be noted that, similar to other CBMC-based detectors for interrupt-driven programs [25, 32, 31], our approach also employs the widely used MiniSat as the underlying solver. However, unlike traditional ones that solely rely on satisfiability solving, we innovatively introduce a customised MAG structure that is functionally equivalent to solving SMT constraints for detecting violations. To spotlight the novelty, the following sections focus on the partial orders required for the graph construction, during which the atomicity violations can be detected precisely and efficiently.

4. Symbolic Encoding-based Identification of Potential Atomicity Violations

In the symbolic encoding (SE)-based potential violations identification as the front-end, an interrupt-driven program is first transformed into its SSA form. Corresponding SMEs are then obtained and partial orders among them are encoded. With this basis, all potential violations are identified, and key read-from orders are further extracted to guide subsequent detection in the back-end.

4.1. Static Single Assignment and Symbolic Memory Event

To achieve the symbolic encoding, an interrupt-driven program is first transformed into its Static Single Assignment (SSA) form. Here, every occurrence of each variable is represented by a uniquely indexed variable only appearing once, irrespective of whether it is a read or write operation. In order of the priority of each task, the value assignments of variables are encoded by interpreting the SSA steps. The corresponding SSA form for program *Exa.c* is illustrated in Fig. 3(b).

Upon transforming the original interrupt-driven program into the corresponding SSA form, the symbolic read and write events are identified within Symbolic Memory Events (SMEs), as shown in Fig. 3(c). This is crucial because only read and write events, which may influence the values of shared variables, are relevant for atomicity violations. As an instance, the SSA

equation $y_4 = x_4$ generates two symbolic memory events: (R_{x_4}) (R x x_4) and (W_{y_4}) (W y y_4). This means that, after event R_{x_4} reads memory location x (i.e. variable x in the original program) with symbolic value x_4 , event W_{y_4} further writes to the memory location y with symbolic value y_4 .

For future convenience, the basic operations related to memory access events are given as follows:

- Function $mloc(e)$ represents the memory location of event e , while $val(e)$ the stored value. For example, events W_{x_0} and W_{x_3} both operate on variable x , resulting in $mloc(W_{x_0}) = mloc(W_{x_3})$, however $val(W_{x_0}) \neq val(W_{x_3})$.
- Function $grd(e)$ means the guard condition for occurrence of event e . For instance, $grd(W_{x_2})$ is $y_1 = 1$ since event W_{x_2} occurs only if the condition $y == 1$ in Line 9 holds.
- Function $hb(e_1, e_2)$ indicates an order that the event e_1 happens before e_2 in any execution of the program, denoted as $e_1 \rightarrow e_2$ for simplicity. For example, $W_{y_3} \rightarrow W_{x_3}$ since W_{y_3} always precedes W_{x_3} in the main task.

4.2. Ordering Encoding

After the SMEs are obtained, the partial orders of events are further encoded, representing the partial order relations over the memory access events of shared variables. Notably, only each *program order* (PG-order) and *read-from order* (RF-order) are explicitly encoded:

- *Program order* (PG-order) is defined as the textual order of memory events within each task. Under the program order, it is assumed that the execution order in any task is consistent with the order of the program.
- *Read-from order* (RF-order) indicates that a valid read event must get its value from a certain write event of the same variable, which means there exists a direct data dependency between the write and the read events.

This is because other orders will be inferred from these encoded orders on-the-fly during the further MAG construction in Sec. 5, so that the explicit encoding of them can be eliminated. Specifically, these orders include the *write-serialisation order* (WS-order) establishing a relative order between writes to the same variable, and the *from-read order* (FR-order) indicating the ordering between a read and subsequent writes.

To mark easily, a Boolean variable $pg(e_j, e_i)$, called *PG*-variable, is used to represent the *PG*-order between memory events e_j and e_i . For example, in the main task of program **Exa.c**, there exists a *PG*-order between R_{x_4} and W_{y_4} , denoted as $pg(R_{x_4}, W_{y_4})$. Similarly, *RF*-variable $rf(e_j, e_i)$ indicates that event e_i reads the value written by e_j . Furthermore, if a read event e_i occurs, it must obtain its value from exactly one write event that accesses the same variable, ensuring the consistency of memory operations. For instance, $rf(W_{x_2}, R_{x_4})$ denotes that the read event R_{x_4} obtains its value from the write event W_{x_2} to the variable x . Intuitively, the constraint inferred from $rf(e_j, e_i)$ is:

$$rf(e_j, e_i) \Rightarrow (val(e_i) = val(e_j)) \wedge grd(e_j) \wedge (e_j \rightarrow e_i)$$

$$grd(e_i) \Rightarrow \bigvee_{mloc(e_i)=mloc(e_j)} rf(e_j, e_i)$$

4.3. Potential Atomicity Violations Identification

Based on the earlier symbolic encoding of the program, potential atomicity violations are identified through the symbolic representation. With this basis, the *RF*-order within each potential violation is extracted as the key *RF*-order, which further plays a guiding role in the subsequent detection.

Algorithm 1: Potential Violations Identification

Input: An interrupt-driven program P

Output: Set $PVSet$ of potential violations with key *RF*-orders

```

1  $Patterns \leftarrow \{(R, W, R), (W, W, R), (W, R, W)\}; PVSet \leftarrow \emptyset;$ 
2  $Events \leftarrow \{(T, \{e_1, \dots, e_n\}) \mid T \text{ is a task, } e_i \text{ is an event in } T\};$ 
3 for  $(T_i, E_i), (T_j, E_j) \in Events$  s.t.  $Pri(T_j) > Pri(T_i)$  do // Get events for
   each two tasks
4   for  $e_1, e_3 \in E_i, e_2 \in E_j$  s.t.  $mloc(e_1) = mloc(e_2) = mloc(e_3)$  do
5     if  $(type(e_1), type(e_2), type(e_3)) \in Patterns$  then
6       if  $type(e_2) = R$  then // For Pattern: (W,R,W)
7          $PVSet \cup = \{(rf(e_1, e_2), (e_1, e_2, e_3))\};$ 
8       else // For Pattern: (R,W,R) and (W,W,R)
9          $PVSet \cup = \{(rf(e_2, e_3), (e_1, e_2, e_3))\};$ 
10 return  $PVSet;$ 

```

To this end, Algorithm 1 formally presents the procedure for identifying potential violations. The algorithm takes an interrupt-driven program as

input and extracts symbolic memory events from each task (Line 2). Then, for every pair of tasks where a high-priority task may preempt a low-priority one (Line 3), it analyzes interleavings that may lead to atomicity violations. Specifically, for any such pair of tasks, the algorithm enumerates event triples that access the same memory location (Line 4) and checks whether their access types match any of the predefined unserializable interleaving patterns (Lines 5–6), as shown in Fig. 1. Once a matching pattern is found, indicating a potential violation, the algorithm extracts the corresponding key RF -order based on the pattern type and records its associated RF -variable along with the violation (Lines 7–9). Eventually, the algorithm returns the $PVSet$ (Line 10), which stores the potential atomicity violations associated with key RF -variables, to be passed to the back-end for further analysis.

Fig. 5 illustrates the key execution steps of Alg. 1 on the example program `Exa.c`. For clarity, a MAG representation is employed in advance, whose detailed concept and construction will be further introduced in the next section. On the left side of the figure, symbolic memory events are obtained from both the main task and the ISR. These events are distinguished based on the shared variables they access, with variable x marked in blue and y in green. Then, the events about each shared variable are attempted to match the unserializable interleaving patterns in Fig. 1. For instance, three events accessing the same shared variable x , W_{x3} and W_{x2} (from the low-priority main task), and R_{x4} (from the high-priority ISR) match **Pattern: (W,W,R)**. Each matched triple indicates a potential atomicity violation. In total, three potential violations are identified, including (W_{x3}, W_{x2}, R_{x4}) , (W_{x3}, W_{x1}, R_{x4}) , and (W_{y3}, R_{y1}, W_{y4}) . Finally, their corresponding key RF -variables, $rf(W_{x2}, R_{x4})$, $rf(W_{x1}, R_{x4})$, and $rf(W_{y3}, R_{y1})$ are extracted and stored in the set $PVSet$, respectively.

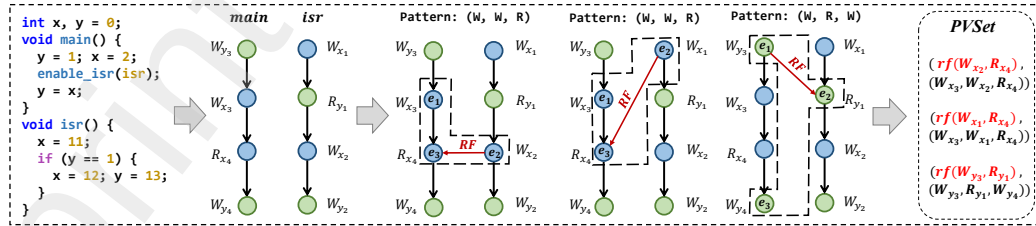


Figure 5: Potential atomicity violations of program `Exa.c`

It is worth emphasizing that the obtained key RF -variables above effectively capture the critical data dependencies that may lead to atomicity

violations. Specifically, when a key RF -variable $rf(e_j, e_i)$ is satisfied, the read event e_i is confirmed to obtain its value from the write event e_j within a violation scenario. With this fact, such key RF -variables can be leveraged as the precise guidance for further detection to concentrate on interleavings that are truly likely to cause violations, thereby improving both efficiency and accuracy.

5. Memory Access Graph-based Confirmation of Real Atomicity Violations

With the previously obtained PG -variables, RF -variables, and $PVSet$ that capture potential atomicity violations associated with key RF -variables, the back-end focuses on confirming real violations. To this end, a dedicated graph structure, called the *Memory Access Graph* (MAG), where nodes correspond to memory access events and edges capture the execution order among them.

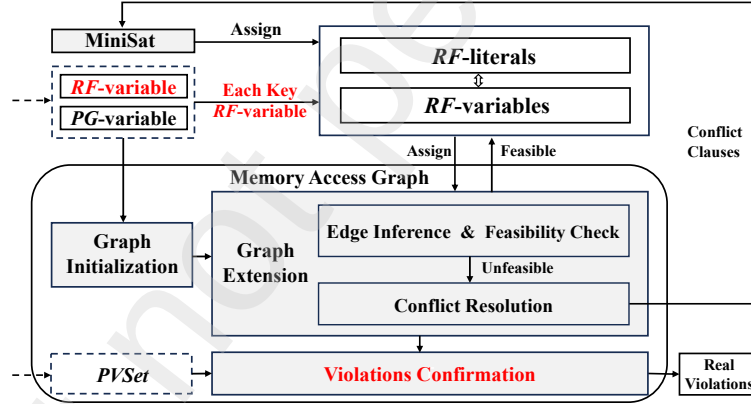


Figure 6: Framework of real atomicity violations confirmation based on MAG

Fig. 6 illustrates the overall framework for the confirmation of real atomicity violations based on the MAG. For each key RF -variable, MiniSat serves as the underlying SAT solver and continues to assign values to other RF -literals corresponding to RF -variables. These assignments are then passed to the MAG for further graph extension, where edge inference is performed based on the current assignment to infer as many edges as possible. Subsequently, a feasibility check is conducted to determine whether to proceed with further assignments or resolve conflicts. Once a valid execution path is

found, it is analyzed with $PVSet$ extracted by the frontend to confirm the real violations.

5.1. Edge Inference Rules

Considering the implicit correlations between RF -orders and other orders, as well as the characteristics of interrupt-driven programs, a set of rules is given in the following so that edges can be inferred effectively during further graph extension.

The first consideration is that WS -orders and FR -orders can be inferred on-the-fly from RF -orders. Specifically, for a write event e_j and read event e_i with $rf(e_j, e_i)$, and another enabled write event e_k accessing the same address (i.e., $mloc(e_j) = mloc(e_i) = mloc(e_k)$), if $rf(e_j, e_i)$ holds, then e_k cannot occur between e_j and e_i . Otherwise, e_i should read the value from e_k , which contradicts $rf(e_j, e_i)$. Therefore, e_k must occur either before e_j or after e_i , leading to the following two rules:

- *Write-Serialization Rule (WS-Rule)*: For any write events e_j and e_k , and a read event e_i , all accessing the same memory address, $rf(e_j, e_i) \wedge (e_k \rightarrow e_i) \wedge grd(e_k) \Rightarrow ws(e_k, e_j)$.
- *From-Read Rule (FR-Rule)*: For any write events e_j and e_k , and a read event e_i , all accessing the same memory address, $rf(e_j, e_i) \wedge (e_j \rightarrow e_k) \wedge grd(e_k) \Rightarrow fr(e_i, e_k)$.

On the other hand, consider the asymmetric preemption and non-blocking execution in interrupt-driven programs, the following rule can be raised:

- *Nested-Interrupt Rule (NI-Rule)*: For two events e_i and e_j within ISRs T_i and T_j , respectively, where e_{l_i} is the last event of T_i , $(e_i \rightarrow e_j) \wedge (Pri(T_i) > Pri(T_j)) \Rightarrow ni(e_{l_i}, e_j)$.

Based on the above rules, the corresponding WS -edge, FR -edge, or NI -edge can be added to the MAG whenever such an order is inferred, as shown in Fig. 7.

5.2. MAG Initialization and RF -guided Extension

At the initialization stage of the MAG construction, all RF -variables are unassigned, and only PG -variables are considered since they are statically determined based on the textual order of memory events within each task.

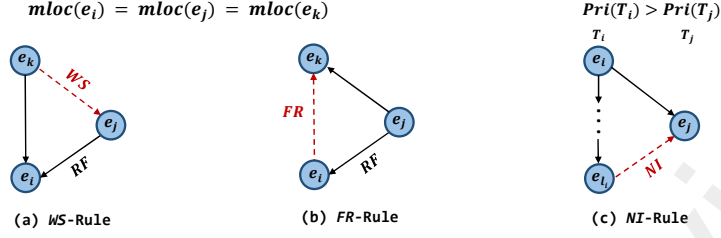


Figure 7: Edge inference rules

Specifically, for each PG -variable $pg(e_j, e_i)$ assigned *true*, a corresponding PG -edge between event e_j and e_i is added to the MAG, establishing the initial PG -order between events. Each time a new edge is added, the transitive closure of the graph is computed to infer indirect orders implied by existing edges, referred to as *transitivity edges*. In this way, all PG -edges and corresponding transitivity edges are generated, forming the graph skeleton.

Following the initialization of the MAG based on PG -variables, its extension is guided by leveraging assignments of RF -variables. In particular, for each key RF -variable, MiniSat serves as the underlying SAT solver and incrementally assigns values to RF -literals corresponding to other RF -variables. Whenever an RF -literal is assigned *true*, the associated RF -edge is generated, and the graph extension is conducted under the above edge inference rules.

Specifically, the *WS-Rule* and *FR-Rule* are first applied to infer *WS* and *FR*-edges, followed by *NI-Rule* to derive *NI*-edges that capture the interrupt-driven program characteristics. Whenever a new edge is added to the graph, a transitive closure is computed to infer additional transitivity edges, thereby refining the ordering between any two events. When no edges can be further inferred, the graph is considered to have reached a stable state. At this point, the feasibility check is performed to determine whether the current execution path contains cyclic dependencies by detecting whether the graph contains a cycle formed by directed edges. If a cycle is detected, the path is deemed invalid. With transitive closure, this check reduces to traversing each node to identify self-loops.

Once a self-loop exists in the MAG, the conflict resolution is triggered. This involves backtracking by undoing the assignments and the inferred edges that cause the conflict, restoring the graph to an acyclic stable state. A conflict clause is then generated and returned to the SAT solver to avoid the same conflict. Otherwise, the solver proceeds by assigning the next unassigned

literal and iterates the extension process. Finally, the extension process terminates when a stable and acyclic MAG has been constructed, indicating a valid program execution path has been found. In this case, the confirmation of real atomicity violations will be performed.

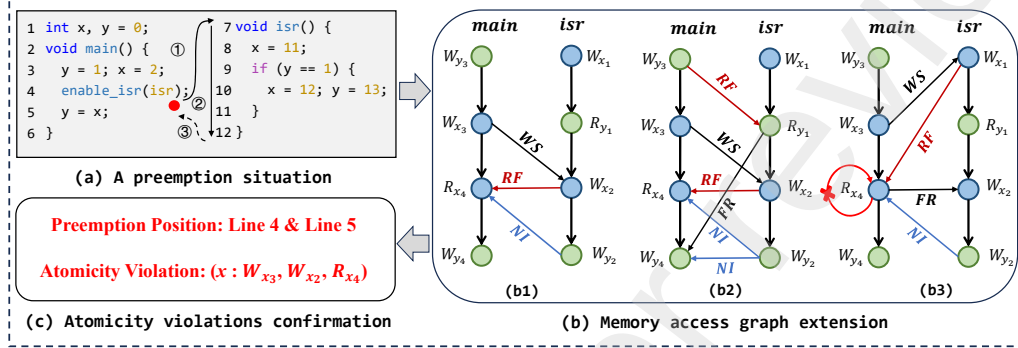


Figure 8: Real atomicity violations confirmation based on MAG for Exa.c

Fig. 8(a) illustrates a preemption scenario in Exa.c, while Fig. 8(b) depicts its corresponding extension on the MAG. For clarity, transitive edges inferred with the transitive closure are omitted. Specifically, if the RF -variable $rf(W_{x_2}, R_{x_4})$ is assigned *true*, the corresponding edge is added to the graph. Based on the *WS-Rule* and *NI-Rule*, two additional edges $ws(W_{x_3}, W_{x_2})$ and $ni(W_{y_2}, R_{x_4})$ are inferred. After employing the transitive closure, the graph reaches a stable state with all derivable edges included, as shown in Fig. 8(b1). Continuing with the assignment $rf(W_{y_3}, R_{y_1})$ *true*, further extension leads to a stable and acyclic MAG, as shown in Fig. 8(b2). In contrast, Fig. 8(b3) illustrates a case where the feasibility check fails, since the transitive closure reveals a self-loop on R_{x_4} .

It is worth noting that although the confirmation of real atomicity violations is performed for each valid program execution path, not every such path surely contains a violation. Therefore, through prioritizing the consideration of key RF -orders, the detection can focus on the paths more likely to contain real violations, thereby reducing the overall state space explored.

5.3. Real Atomicity Violations Confirmation

Whenever the MAG extension is finished, the confirmation of real atomicity violations is performed on this basis. Here, each assigned key RF -variable is leveraged to locate the corresponding potential violation and confirm it with the following rule.

Specifically, for a potential violation represented as a triplet (e_1, e_2, e_3) , if its pattern type is **Pattern**:(R,W,R) or **Pattern**:(W,W,R), the key *RF*-variable links the write event e_2 in the high-priority task to the read event e_3 in the low-priority task. Given this, the backward search starts from e_3 to find the nearest feasible access e_1 to the same variable within the task. During this process, the following two conditions must be met to confirm a real atomicity violation:

- **Proximate Location**: e_1 is the nearest previous event accessing the same memory as e_3 . This is because if an intermediate event exists between them, the proximity between e_1 and e_3 is disrupted, rendering the violation condition invalid.
- **Ordering Edge**: An edge must exist from event e_1 to e_2 in the MAG. This is because if such an ordering edge is absent, the execution order of e_1 and e_2 cannot be determined, invalidating the violation condition.

Similarly, if a potential violation triplet (e_1, e_2, e_3) is of the type **Pattern**:(W,R,W), it can also be located in the MAG via its key *RF*-variable, followed by the two conditions mentioned above. In this case, since the key *RF*-edge is from event e_1 to e_2 , the ordering edge check will validate whether there exists an edge from e_2 to e_3 in the MAG.

As an example, for the stable and acyclic MAG in Fig. 8(b2), the key *RF*-variable $rf(W_{x_2}, R_{x_4})$ locates a potential violation of **Pattern**:(W,W,R). Since W_{x_3} is the nearest feasible write event before R_{x_4} , the location proximity holds. Furthermore, since the edge inference rules infer $W_{x_3} \rightarrow W_{x_2}$, the ordering edge check passes. Therefore, $(W_{x_3}, W_{x_2}, R_{x_4})$ is confirmed as a real atomicity violation shown in Fig. 8(c).

6. Experiment and Evaluation

The experiments are designed to answer the following research questions (RQs):

RQ1: How is BMC4AV in detecting atomicity violations compared to SOTA tools, including intAtom [23], CPA4AV [24], NIChecker [25], and the specially extended version of iCBMC [33, 32]?

RQ2: What is the contribution of the proposed *RF*-guidance strategy to both of the precision and efficiency?

RQ3: What is the contribution of the proposed MAG-based detection strategy to the efficiency?

6.1. Implementation and Experimental Setup

The proposed approach has been implemented as a tool named BMC4AV, designed to detect atomicity violations in interrupt-driven C programs. Specifically, BMC4AV is developed based on CBMC [26] and MiniSat [27], where CBMC is a powerful bounded model checker employed as the front-end, while MiniSat is an efficient SAT solver employed as the back-end. In addition, one MAG is maintained to achieve effective detection. Two benchmarks employed in the experimental evaluation are described as follows:

- Racebench 2.1: an open-source academic benchmark² consisting of 31 interrupt-driven C programs based on real-world embedded aerospace software. It has been widely employed to evaluate SOTA tools. Although short, these cases exhibit non-trivial structures and cover diverse factors of atomicity violations, such as interleaving patterns, shared data types, access modes, and control flows, providing sufficient complexity to evaluate the completeness of BMC4AV. A total of 25 test cases are selected for the experiment, with 6 cases excluded because they only contain the access interleaving pattern (R,W,W), which is considered as a kind of benign atomicity violations.

- Real-world Programs: the benchmark set³ employed by NIChecker to detect real-world atomicity violations. This benchmark is derived from 18 interrupt-driven programs originating from six real-world embedded software packages. These programs range in size from a few hundred to over a thousand lines of code, and can be employed to fully evaluate the scalability of BMC4AV.

For a fair evaluation, all loops in the above benchmarks are abstracted using the same loop abstraction strategy [34, 35] in NIChecker, which is the latest detector published in this year for atomicity violations of interrupt-driven programs. All experiments are performed on a computer running the Ubuntu 20.04 LTS system with an Intel(R) Xeon(R) E5-2620 CPU and 32 GB RAM. This computer is identical to the one used for evaluating intAtom but is less powerful than the one for NIChecker. All reported time is wall-clock time, measured in seconds with the Unix time command.

6.2. Comparison with SOTA Tools

To answer RQ1, we perform the evaluation of BMC4AV in detecting atomicity violations on Racebench 2.1 and real-world programs by comparing

²<https://github.com/chenruihua/racebench>

³<https://github.com/zhuangyuan/NIChecker>

it with SOTA tools.

6.2.1. Evaluation on Racebench 2.1

Although a comprehensive comparative analysis of SOTA tools Rchecker, intAtom, CPA4AV, and NIChecker is intended, not all of them are available. For example, Rchecker, intAtom, and NIChecker are not open-source. Fortunately, all of these tools have been evaluated using Racebench 2.1, and their experimental data are publicly available. In addition, the comparison between intAtom and Rchecker has been conducted, and both the precision and efficiency of intAtom are higher than those of Rchecker. Therefore, the experimental data reported in their published papers are used, and BMC4AV is compared only to intAtom, CPA4AV and NIChecker. Although the experimental configurations differ across tools, the comparison remains valid, since BMC4AV is evaluated on the same configuration as intAtom, which is weaker than that used by NIChecker, thereby ensuring the fairness of the results.

Additionally, to make the evaluation more convincing, we adapt and extend the pioneering verification tool iCBMC for interrupt-driven programs, named iCBMC⁺, and compare it with BMC4AV. The reason for selecting iCBMC⁺ as a baseline is its CBMC-based path feasibility solving strategy similar to BMC4AV. This ensures the observed improvements result from our proposed approach. Considering that iCBMC is originally a tool for detecting assertion violations, the modification focuses on extending the front-end to implement a potential violations identification process similar to that of BMC4AV, encoding the identified violations as assertions, and then solving them with its native solver to detect atomicity violations.

Table 1 shows the experimental results of BMC4AV and other SOTA tools. Columns 1-3 list the index (#ID), lines of code (#LoC), and number of atomicity violations (#Vio). #WN represents the number of atomicity violations warnings detected by each tool. #TP and #FP represent the number of true positives and false positives, respectively. In addition, we employ precision ($\#TP/\#WN$) as an evaluation metric. The failure of a detection tool can be attributed to three possible factors: lack of support (-), timeout (T.O.), or out of memory (O.O.M.).

The data show that although iCBMC⁺, intAtom and NIChecker detect all real violations, they produce 20, 6, and 2 false positives, while BMC4AV detects all 38 real atomicity violations without any false positives. This improvement is attributed to the ability of our tool to uncover the underly-

Table 1: Comparing BMC4AV with SOTA tools on Racebench 2.1

#ID	#LoC	#Vio	iCBMC ⁺				intAtom				CPA4AV				NIChecker				BMC4AV			
			#WN	#TP	#FP	#T(s)	#WN	#TP	#FP	#T(s)	#WN	#TP	#FP	#T(s)	#WN	#TP	#FP	#T(s)	#WN	#TP	#FP	#T(s)
1	70	1	2	1	1	5.38	2	1	1	0.20	- ¹	-	-	-	1	1	0	13.54	1	1	0	1.96
2	52	1	2	1	1	6.24	2	1	1	0.09	-	-	-	-	1	1	0	4.42	1	1	0	2.02
3	79	1	2	1	1	4.60	1	1	0	0.48	0	0	0	T.O. ²	1	1	0	8.74	1	1	0	0.31
4	74	1	1	1	0	4.03	1	1	0	0.38	1	1	0	0.41	1	1	0	10.48	1	1	0	0.22
5	52	1	1	1	0	2.30	1	1	0	0.13	0	0	0	T.O.	1	1	0	3.97	1	1	0	0.26
6	59	1	1	1	0	5.83	1	1	0	0.27	1	1	0	0.98	1	1	0	11.44	1	1	0	0.68
7	56	1	3	1	2	6.49	1	1	0	0.33	-	-	-	-	1	1	0	3.51	1	1	0	0.24
8	57	1	2	1	1	3.41	2	1	1	0.38	-	-	-	-	1	1	0	3.70	1	1	0	0.26
9	52	1	4	1	3	8.75	1	1	0	0.16	-	-	-	-	1	1	0	9.90	1	1	0	0.20
10	58	1	2	1	1	3.26	2	1	1	0.13	-	-	-	-	2	1	1	6.42	1	1	0	0.16
11	48	1	2	1	1	6.31	1	1	0	0.08	-	-	-	-	2	1	1	13.34	1	1	0	0.22
12	37	1	1	1	0	3.21	1	1	0	0.03	-	-	-	-	1	1	0	3.11	1	1	0	0.16
13	71	1	2	1	1	3.58	2	1	1	0.20	-	-	-	-	1	1	0	7.91	1	1	0	0.25
14	64	1	2	1	1	3.57	2	1	1	0.16	-	-	-	-	1	1	0	7.63	1	1	0	0.22
15	45	1	1	1	0	2.24	1	1	0	0.09	1	1	0	0.70	1	1	0	6.64	1	1	0	0.18
16	38	3	3	3	0	7.19	3	3	0	0.06	1	1	0	0.66	3	3	0	6.70	3	3	0	0.20
17	49	3	4	3	1	6.61	3	3	0	0.11	0	0	0	O.O.M. ³	3	3	0	13.99	3	3	0	0.30
18	69	3	3	3	0	3.58	3	3	0	0.11	-	-	-	-	3	3	0	6.91	3	3	0	0.22
19	79	1	3	1	2	5.58	1	1	0	0.53	-	-	-	-	1	1	0	10.95	1	1	0	0.29
20	60	2	2	2	0	2.34	2	2	0	0.17	-	-	-	-	2	2	0	7.13	2	2	0	0.23
21	86	2	3	2	1	13.41	2	2	0	0.58	1	1	0	2.92	2	2	0	11.46	2	2	0	0.44
22	77	4	4	4	0	8.34	4	4	0	0.22	2	2	0	5.13	4	4	0	10.37	4	4	0	0.34
23	44	1	1	1	0	3.23	1	1	0	0.06	0	0	0	0.20	1	1	0	6.65	1	1	0	0.22
24	68	1	2	1	1	4.53	1	1	0	0.14	-	-	-	-	1	1	0	3.77	1	1	0	0.52
25	97	3	3	3	0	6.35	3	3	0	0.21	-	-	-	-	3	3	0	3.81	3	3	0	0.34
Total	1541	38	58	38	20	130.36	44	38	6	5.30	7	7	0	-	40	38	2	196.49	38	38	0	10.44
Precision			67.9%				86.4%				-				95%				100%			

-¹: lack of supportT.O.²: Time Out 100sO.O.M.³: Out of Memory 2GB.

ing relation between atomicity violations and *RF*-orders, and further utilize this relation to guide the precise detection. In contrast, intAtom relies on coarse-grained reachability analysis based on control flow graphs, making false positives inevitable, while NIChecker only provides coarse descriptions for compound data types, leading to false positives in specific cases. As for iCBMC⁺, although its front-end identifies potential violations as BMC4AV, its encoding stage fails to fully exploit the correlation between key *RF*-orders and potential violations, resulting in incorrectly labelling interleavings in complex paths that actually contain no violations. Furthermore, based on CPAChecker, the inherent reachability computation in CPA4AV relies on a binary decision diagram or explicit-value analysis, resulting in its timeouts or memory overflows in #ID 3, 5 and 17.

In terms of efficiency, BMC4AV completes all detection tasks in 10.44 seconds, which is significantly lower than the 130.36 seconds needed by iCBMC⁺ and the 196.49 seconds needed by NIChecker. This is because NIChecker requires a substantial amount of time for front-end serialization to transform an interrupt-driven program into a bounded sequential C program and can only detect one type of atomicity violation pattern for a single variable in each detection run, necessitating multiple detection runs for a single example. The inefficiency of iCBMC⁺ stems partly from an unoptimized explicit encoding

that introduces a large number of redundant partial-order constraints, greatly increasing formula complexity and solver load. Moreover, during verification, iCBMC⁺ terminates once a violation is found, requiring manual adjustment and reruns to explore other paths, which is time-consuming. As for CPA4AV, besides the three examples where it runs out of time or memory, the time taken for the other seven ones is much higher than that of BMC4AV.

6.2.2. Evaluation on Real-world Programs

To evaluate the scalability of BMC4AV, real-world programs are further detected. Considering that intAtom and Rchecker are not open-source and have not been evaluated on these programs, BMC4AV is compared only with the open-source tools iCBMC⁺, CPA4AV, and the latest tool NIChecker. As shown in Table 2, besides the same columns as Table 1, the hit rate ($\#TP/\#Vio$) represents the proportion of actual violations correctly detected, and $\#FN$ represents the number of false negatives.

Table 2: Comparing BMC4AV with SOTA tools on real-world programs

#Name	#LoC	#Vio	iCBMC ⁺					CPA4AV					NIChecker					BMC4AV				
			#WN/ #TP	#FP	#FN	#T(s)	#Mem (MB)	#WN/ #TP	#FP	#FN	#T(s)	#Mem (MB)	#WN/ #TP	#FP	#FN	#T(s)	#Mem (MB)	#WN/ #TP	#FP	#FN	#T(s)	#Mem (MB)
logger1	158	2	4/2	2	0	8.17	31.05	6/6	0	0	1.84	252.68	0/0	0	1	14.12	29.38	2/2	0	0	0.30	17.69
logger2	183	0	0/0	0	0	4.52	28.56	2/0	1	3	3.19	240.75	0/0	0	0	7.41	29.86	0/0	0	0	0.34	17.97
logger3	188	2	3/2	1	0	8.08	47.30	5/4	1	2	2.78	354.59	1/1	0	1	7.40	30.46	2/2	0	0	0.28	17.91
blink1	149	2	2/2	0	0	7.26	31.63	6/6	0	0	1.69	218.13	2/2	0	0	13.28	29.89	2/2	0	0	0.22	16.25
blink2	161	3	3/3	0	0	8.51	33.90	3/3	0	2	2.12	277.44	2/2	0	1	13.80	30.51	3/3	0	0	0.21	16.64
blink3	179	3	3/3	0	0	11.90	47.44	12/12	0	0	3.15	361.27	3/3	0	0	14.04	31.47	3/3	0	0	0.24	17.04
brake1	517	2	4/2	2	0	21.27	290.44	0/0	0	0	T.O.	-	2/2	0	0	12.95	201.08	2/2	0	0	2.18	65.95
brake2	623	3	5/3	2	0	19.73	132.35	9/9	0	0	4.12	503.04	3/3	0	0	13.72	36.25	3/3	0	0	0.87	21.97
brake3	770	2	9/2	7	0	86.98	357.48	0/0	0	0	T.O.	-	2/2	0	0	21.26	216.37	2/2	0	0	5.14	68.98
i2c_pca_isa_1	363	1	1/1	0	0	8.38	67.10	-	-	-	-	-	1/1	0	0	11.17	30.70	1/1	0	0	0.33	17.83
i2c_pca_isa_2	379	4	5/4	1	0	5.74	76.55	6/6	1	0	3.21	339.61	2/2	0	2	11.95	32.52	4/4	0	0	0.26	17.75
i2c_pca_isa_3	415	8	26/8	18	0	178.32	545.63	-	-	-	-	-	4/4	0	4	18.30	198.46	8/8	0	0	2.61	64.91
i8xx_tco_1	813	3	6/3	3	0	17.51	117.46	-	-	-	-	-	3/3	0	0	18.48	231.59	3/3	0	0	2.32	64.85
i8xx_tco_2	976	3	4/3	1	0	24.68	38.58	-	-	-	-	-	3/3	0	0	15.59	36.96	3/3	0	0	0.28	17.96
i8xx_tco_3	963	2	3/2	1	0	23.54	38.76	-	-	-	-	-	2/2	0	0	16.99	37.98	2/2	0	0	0.27	17.66
wdt_pci_1	1266	10	14/10	4	0	26.74	57.85	-	-	-	-	-	2/2	0	8	14.63	36.20	10/10	0	0	0.40	18.47
wdt_pci_2	1174	15	21/15	6	0	125.35	465.44	0/0	0	0	T.O.	-	2/2	0	13	15.01	213.08	15/15	0	0	2.77	65.89
wdt_pci_3	1356	29	53/29	24	0	157.87	121.46	0/0	0	0	-	O.O.M.	3/3	0	26	22.29	40.32	29/29	0	0	2.10	21.49
Total	10633	94	166/94	72	0	744.55	2528.98	49/36	3	10	-	-	37/37	0	57	262.39	1493.08	94/94	0	0	21.12	567.21
Precision			56.3%										100%					100%				
Hit rate				100%										39.4%					100%			

After removing the Pattern: (R,W,W), which is not considered an atomicity violation, NIChecker detects 37 violations in a total of 18 programs. It should be pointed out that, all of these 37 violations are manually injected by the authors of NIChecker. However, after our in-depth analysis, the actual number of atomicity violations in these programs is 94, indicating that NIChecker misses 57 violations (i.e., false negatives). The main reason is that NIChecker relies on manually inputting patterns of atomicity violations for global variables, and only the predesigned variables can be considered.

Since NIChecker does not have prior knowledge of which variables may lead to atomicity violations, a large number of violations are missed. Furthermore, it is discovered that even if all possible patterns for each variable are input, NIChecker still cannot detect all violations, because its strategy involves adding auxiliary code to transform the violations detection into the assertions detection. This means that for a large-scale program with numerous variables and read/write statements, it is not guaranteed that all auxiliary code blocks can be inserted at the correct locations. Due to these reasons, the hit rate of NIChecker is only 39.4%.

For iCBMC⁺, although all 94 genuine atomicity violations are detected, it produces 72 false positives, yielding a false positive rate of 43.4%. In particular, programs such as *i2c_pca_isa_3* and *wdt_pci_3* produce 18 and 24 false positives, respectively. The root cause, same with its imprecision on Racebench 2.1, is that iCBMC⁺ fails to leverage key *RF*-orders correlations, making it difficult to accurately detect atomicity violations in complex programs. As for CPA4AV, after excluding 10 unsupported or failed cases (timeouts or out-of-memory), it reports 49 violations, among which 3 are false positives due to imprecise handling of complex data structures (e.g., pointers and structs) that introduce infeasible paths. In addition, its depth-limited abstract reachability tree leads to 7 missed real violations. In contrast, BMC4AV detects all 94 genuine violations without any false positives, demonstrating substantially higher detection precision than iCBMC⁺, CPA4AV and NIChecker.

It should be noted that, although there is currently no ground truth about the violations in these real-world programs, we regard that there is no false negative ($\#FN = 0$) in BMC4AV. This is because, on the one hand, the benchmark is designed and employed by NIChecker, and BMC4AV successfully detects all the violations reported by NIChecker without missing any; on the other hand, in the ordering encoding phase, Alg. 1 conservatively identifies all potential violations through pattern matching, and the MAG only prunes infeasible cases, all of which are manually validated.

In terms of detection time, iCBMC⁺ and NIChecker require 744.55 and 262.39 seconds respectively, while BMC4AV finishes in only 21.12 seconds, reducing the runtime by 97.2% compared to iCBMC⁺ and by 92.0% compared to NIChecker. The inefficiency of iCBMC⁺ stems partly from its unoptimized encoding, which introduces many redundant constraints and greatly increases solving time. Moreover, once a violation is found, users must manually remove the failed assertion and rerun the tool. For example, programs

such as *i2c_pca_isa_3*, *wdt_pci_2*, and *wdt_pci_3* are especially costly due to complex structures and branching paths, generating numerous inter-dependent assertions and requiring 461.54 seconds. The primary reason for the inefficiency of NIChecker is that a significant portion of the runtime is dedicated to lazy sequentialization. In contrast, BMC4AV directly encodes the program symbolically and further accelerates the detection using a customized MAG. Additionally, NIChecker detects only one violation pattern for a single variable in each run, while BMC4AV can detect all violations through a single detection process.

In addition to the time comparison above, in terms of memory usage, iCBMC⁺ and NIChecker require 2528.98 MB and 1493.08 MB respectively, while BMC4AV uses only 567.21 MB. The excessive memory usage of iCBMC⁺ is mainly due to its constraint generation process retaining large amounts of irrelevant intermediate symbols and paths, resulting in a huge set of redundant variables in the formulas. As for NIChecker, it adds numerous auxiliary code segments to enable violation detection, greatly increasing the number of encoded variables in the formula and consequently leading to excessive memory consumption. In contrast, BMC4AV employs a MAG to avoid encoding redundant partial orders, thereby significantly reducing memory usage.

6.3. The Contribution of *RF*-guidance Strategy

To answer RQ2, we focus on the impact of the *RF*-guidance strategy on the activation of *RF*-edges during the detection process. To achieve this, two variants based on BMC4AV are implemented: One variant without the *RF*-guidance strategy, called BMC4AV-NRF, and the other variant that assigns values to all *RF* variables, called BMC4AV-ARF.

The experimental results of evaluation on the real-world programs are shown in Table 3. Compared to the variants, BMC4AV not only detects all atomicity violations but also requires only 21.12 seconds and 567.21 MB of memory. Although BMC4AV-NRF detects 31 true atomicity violations in 20.01 seconds, there are 63 false negatives. In addition, BMC4AV-ARF can also detect all atomicity violations, but the time is 368.17 seconds and the memory usage is 677.70 MB, both significantly higher than BMC4AV. These results indicate that the proposed *RF*-guidance strategy can improve the detection precision while maintaining high efficiency.

To further explore the essential reasons of the *RF*-guidance strategy contribution to the solving process, activated *RF*-edges (#Act. RF) and activated key *RF*-edges (#Key RF) are recorded and shown in Table 3. Specifi-

Table 3: Comparing BMC4AV with BMC4AV-NRF, BMC4AV-ARF and BMC4AV-NG on real-world programs

#Name	#Vio	BMC4AV						BMC4AV-NRF						BMC4AV-ARF						BMC4AV-NG					
		#WN/ #FN	#T(s)	#BT(s)	#Mem (MB)	#Act. RF	#Key RF	#WN/ #FN	#T(s)	#Mem (MB)	#Act. RF	#Key RF	#WN/ #FN	#T(s)	#Mem (MB)	#Act. RF	#WN/ #FN	#T(s)	#Mem (MB)	#WN/ #FN	#T(s)	#Mem (MB)			
logger1	2	2/0	0.30	0.02	17.69	17	2	0/2	0.30	17.76	18	1	2/0	1.01	19.52	51	2/0	0.64	22.45						
logger2	0	0/0	0.34	0.03	17.97	25	3	0/0	0.27	17.77	33	2	0/0	2.09	20.18	61	0/0	0.73	23.42						
logger3	2	2/0	0.28	0.03	17.91	26	3	0/2	0.28	17.96	27	1	2/0	3.05	20.85	81	2/0	0.65	24.91						
blink1	2	2/0	0.22	0.01	16.25	5	5	0/2	0.17	16.47	3	2	2/0	0.54	17.49	24	2/0	0.25	19.94						
blink2	3	3/0	0.21	0.02	16.64	15	5	1/2	0.19	16.52	12	1	3/0	1.18	18.86	18	3/0	0.34	20.85						
blink3	3	3/0	0.24	0.02	17.04	30	8	3/0	0.23	16.95	12	3	3/0	3.02	19.94	35	3/0	0.80	22.67						
brake1	2	2/0	2.18	0.07	65.95	30	7	0/2	2.39	65.80	40	1	2/0	9.34	68.25	71	2/0	4.20	73.92						
brake2	3	3/0	0.87	0.12	21.97	99	15	1/2	0.83	21.56	114	8	3/0	25.61	31.41	173	3/0	2.11	33.51						
brake3	2	2/0	5.14	0.26	68.98	174	46	2/0	3.48	68.94	261	14	2/0	88.26	75.55	315	2/0	7.34	80.86						
i2c_pca_isa_1	1	1/0	0.33	0.03	17.83	42	2	0/1	0.23	17.72	41	0	1/0	9.24	23.64	107	1/0	0.62	25.80						
i2c_pca_isa_2	4	4/0	0.26	0.03	17.75	63	5	0/4	0.24	17.56	55	1	4/0	2.63	23.52	143	4/0	1.08	30.08						
i2c_pca_isa_3	8	8/0	2.61	0.08	64.91	67	12	0/8	3.64	65.23	67	6	8/0	16.41	67.98	151	8/0	4.47	80.02						
i8xx_tco_1	3	3/0	2.32	0.04	64.85	38	6	3/0	2.48	64.71	29	2	3/0	3.76	65.23	91	3/0	2.31	71.89						
i8xx_tco_2	3	3/0	0.28	0.03	17.96	47	6	1/2	0.30	17.96	33	1	3/0	1.62	20.71	76	3/0	0.67	24.55						
i8xx_tco_3	2	2/0	0.27	0.02	17.66	23	6	0/2	0.29	17.74	29	1	2/0	2.41	21.02	95	2/0	0.59	24.19						
wdt_pci_1	10	10/0	0.40	0.04	18.47	49	16	3/7	0.37	18.70	82	7	10/0	0.87	23.87	145	10/0	1.12	28.70						
wdt_pci_2	15	15/0	2.77	0.09	65.89	46	33	5/10	3.08	66.07	129	12	15/0	51.57	72.67	514	15/0	4.69	80.17						
wdt_pci_3	29	29/0	2.10	0.22	21.49	265	78	12/17	1.24	21.41	228	31	29/0	145.56	67.01	403	29/0	13.36	51.81						
Total	94	94/0	21.12	1.16	567.21	1061	258	31/63	20.01	566.83	1213	94	94/0	368.17	677.70	2554	94/0	45.97	739.74						

cally, the activated (key) *RF*-edges refer to the number of (key) *RF*-variables assigned values during the solving process, respectively.

Compared to BMC4AV-NRF, BMC4AV shows a significant advantage in the number of key *RF*-edges in all programs (94 *vs.* 258), meanwhile the total number of activated *RF*-edges is slightly less (1213 *vs.* 1061). This result indicates that the *RF*-guidance strategy can precisely direct the focus to *RF*-edges which are crucial for violation detection, thereby improving the precision without introducing additional overhead. As for BMC4AV-ARF, the number of key *RF*-edges is omitted in Table 3, since all *RF*-variables are assigned. Although BMC4AV-ARF does not have false negatives, the extension of the MAG needs to consider all *RF*-edges, resulting in significant time and memory overhead. From the result, it can be concluded that redundant exploration paths can be effectively avoided with the proposed *RF*-guidance strategy, thereby significantly improving the efficiency and reducing the memory usage, meanwhile the precision can be guaranteed.

6.4. The Contribution of Memory Access Graph-based Detection Strategy

To answer RQ3, a variant of BMC4AV, called BMC4AV-NG, is implemented without constructing a MAG. Due to the inability to infer *WS* and *FR*-edges on the MAG, BMC4AV-NG has to generate corresponding order constraints directly. Table 3 shows the comparison of BMC4AV-NG and BMC4AV on real-world programs. Although BMC4AV-NG has the same precision and hit rate as BMC4AV, it totally takes 45.97 seconds and 739.74

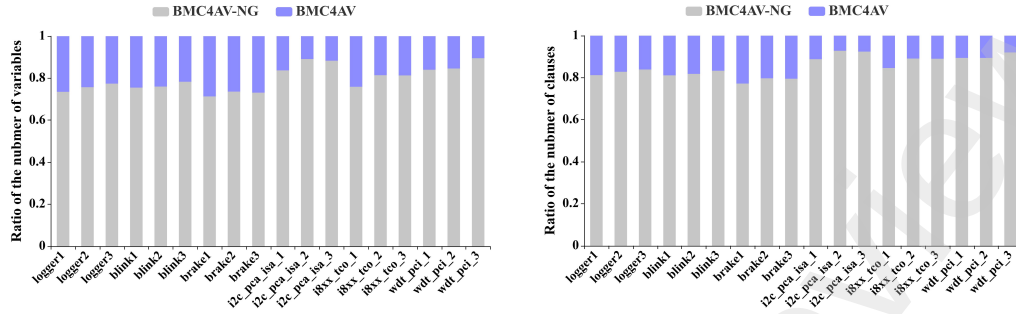


Figure 9: Size of SAT formulas in BMC4AV-NG and BMC4AV

MB of memory, while BMC4AV only 21.12 seconds and 567.21 MB. With the reduction of 54.1% in time and 23.3% in memory, it can be concluded that the MAG-based strategy is effective to improve efficiency and reduce memory usage.

Further, a detailed analysis is conducted to explore the underlying reasons for the MAG contribution. Specifically, the ratio of the number of variables and clauses in SAT formulas generated by BMC4AV and BMC4AV-NG is recorded, as presented in Fig. 9. We can see that BMC4AV generates significantly smaller formulas than BMC4AV-NG. Meanwhile, the additional build time (column “#BT”) taken by BMC4AV for the graph construction shown in Table 3 is only 1.16 seconds.

The above experimental results indicate that BMC4AV-NG generates a large number of redundant order constraints during the encoding, which may not actually take effect for the violations detection. As a result, the complexity of the encoding formula is increased inevitably, which further has a negative impact on the solving efficiency. On the contrary, the proposed MAG-based detection strategy is able to avoid redundant front-end encoding and only derive necessary orders on-the-fly during the MAG construction. In this way, BMC4AV can reduce the memory usage and improve the efficiency of the atomicity violations detection effectively.

6.5. Threats to Validity

The main threats to validity include whether the performance improvements are mainly attributable to our proposed strategies, the credibility of implementation and experimental results.

Firstly, BMC4AV is built on top of the widely-used model checker CBMC and the efficient SAT solver MiniSat. In the front-end, by omitting the ex-

explicit encoding of abundant constraints, BMC4AV adopts a simpler encoding than the original CBMC. In the back-end, a MAG is maintained to only infer necessary orders to confirm potential violations. To evaluate the effectiveness of each strategy, three variants are implemented and ablation experiments are conducted. The experimental results indicate that the performance improvements largely stem from our proposed strategies.

Secondly, all benchmarks in the experiment come from academic and real-world programs, which have been widely accepted by the community for verifying the safety of interrupt-driven programs. In addition, the competitor tools in our experiments are all developed within the past three years. Especially, NIChecker was just released this year. Moreover, the comparison with iCBMC⁺ confirm that the observed improvements stem from our methodological innovations, rather than differences in the basic verification paradigms. According to the detailed experimental results and in-depth analysis, we are confident about the detection precision and efficiency of BMC4AV.

Finally, considering BMC4AV is founded on bounded model checking, in our experiments, all loops are abstracted using the same loop abstraction strategy as NIChecker to ensure completeness. Although there is currently no ground truth about the atomicity violations for these real-world programs, it is believed that there is no false negative in BMC4AV, since these programs are designed by NIChecker, and BMC4AV successfully detects all violations without missing any. In addition, to further ensure the credibility of the results, we adopt an over-approximation approach, where Alg. 1 conservatively identifies all potential violations with pattern matching, and the MAG only prunes infeasible cases, all of which are manually validated as non-violations.

7. Related Work

As one kind of serious defects, atomicity violations usually occur in concurrent programs, including multi-threaded and interrupt-driven ones.

7.1. Atomicity Violation Detection for Multi-threaded Programs

Among dynamic approaches, Ma et al. [36] manage a streamlined set of increasing dependency sequences for each active transaction to uncover violations in a trace. Eslamimehr et al. [1] directly identify instances of three-access patterns during dynamic analysis. By analyzing happens-before relations in an execution trace, Chang et al. [37] detect event pairs meant to

be processed atomically and use predefined patterns to uncover violations. In terms of static detection, Dias et al. [21] introduce a method with analyzing the dependency graph of program variables to identify atomicity violations. Flanagan et al. [38, 39] and Sasturkar et al. [40] propose distinct type systems to define method atomicity and detect violations in multi-threaded Java programs. Through a combination of static and dynamic analyses, Chen et al. [19] extract summaries during static analysis and instantiate them using runtime values. Wang et al. [41] first identify atomicity violations based on prediction, and then monitor execution interleavings to record traces. Kasikci et al. [42] integrate dynamic and static inter-procedural pointer and type analysis to uncover root causes of concurrency bugs.

Overall, although the space reduction strategies such as partial order reduction have been proposed, they have not effectively captured the characteristics of thread interleavings. In contrast, interrupt-driven programs have a smaller state space, making static analysis more feasible. However, interrupt-driven programs differ significantly from multi-threaded ones in terms of synchronization mechanisms and preemption relations, making it challenging to directly apply static analysis techniques designed for multi-threaded programs to interrupt-driven ones.

7.2. Atomicity Violation Detection for Interrupt-driven Programs

In recent years, atomicity violations have emerged as significant issues in interrupt-driven programs. Similarly based on the bounded model checker CBMC, Du et al. [31, 43] develop a static analysis tool Rchecker, targeting the detection of atomicity violations. However, the intrinsic characteristics of path constraint resolution inevitably lead to false positives. Additionally, the lack of effective path pruning strategies results in an enormous state space. intAtom [23] applies a staged strategy to first detect potential violations using data-flow analysis and then filter out infeasible candidates. Despite this, false positives arise due to the inaccuracy of reachability analysis. CPA4AV [24] performs the reachability computation with an abstract reachability tree, making it not fully support complex data types and not efficient. As the latest detector, NIChecker [25] transforms an interrupt-driven program into a bounded sequential one using lazy sequentialization. Whereas, its efficiency is relatively low because the front-end serialization requires significant time, and only one pattern for one variable can be detected in each run, thus preventing fully automated verification. In addition, the employed assertion-insertion strategy may introduce false negatives due to

semantic misinterpretation, leading to imprecise detection. As a pioneering bounded model checking tool for interrupt-driven programs, iCBMC is developed by Kroening et al. [32, 33] originally for assertion verification. Built upon a CBMC-based path feasibility solving framework, it can also be extended to detect atomicity violations. Considering this, we adapt and extend it as a baseline for comparison. However, since it still relies on unoptimized explicit encodings, a large number of redundant constraints are introduced, reducing the detection efficiency. Moreover, when it is applied to complex programs, many false positives are also produced.

To address the limitations of the existing tools in both precision and efficiency, BMC4AV innovatively introduces an approach that leverages a partial order-guided memory access graph. Specifically, a memory access graph is maintained to represent the concrete relations among memory events. Unlike traditional methods that encode all interleavings explicitly as constraints, our approach leverages the implicit correlations within partial orders, which enables us to replace such explicit and redundant encodings with a set of inference rules. Furthermore, guided by the key read-from orders, the MAG can construct a minimal yet sufficient representation of memory event relations for violation detection. This not only ensures accuracy but also achieves a significant improvement in efficiency.

8. Conclusion and Future Work

In this paper, we focus on bounded model checking for atomicity violations in interrupt-driven programs. The key innovation lies in one precise and efficient detection strategy based on a partial order-guided memory access graph. With this strategy, redundant constraints in the encoding can be effectively reduced by exploiting the implicit correlations within partial orders. Moreover, guided by key read-from orders, a minimal yet sufficient representation of memory event relations are constructed in the memory access graph for violation detection. Extensive experiments have been conducted and the results demonstrate that the proposed approach significantly reduces the detection time and exploration space while maintaining precision.

In future work, efforts will be made to explore more precise characterization of harmful atomicity violations. Additionally, a broader range of real-world interrupt triggering mechanisms will be incorporated to enhance the practical applicability.

9. Data Availability

The tools and benchmarks that support this study are available on Figshare at <https://figshare.com/s/2fbd2c948c9722bc5c59>.

Acknowledgment

This research is supported by the National Natural Science Foundation of China with Grant No. 62302375, 62402372, 62372347, 62202371, 62202361 and 62192734. Bin Yu is the corresponding author.

References

- [1] M. Eslamimehr, M. Lesani, and G. Edwards, “Efficient detection and validation of atomicity violations in concurrent programs,” *Journal of Systems and Software*, vol. 137, pp. 618–635, 2018.
- [2] Y. Sun, S.-C. Cheung, S. Guo, and M. Cheng, “Disclosing and locating concurrency bugs of interrupt-driven IoT programs,” *IEEE Internet of Things Journal*, vol. 6, no. 5, pp. 8945–8957, 2019.
- [3] J. Wang, W. Dou, Y. Gao, C. Gao, F. Qin, K. Yin, and J. Wei, “A comprehensive study on real world concurrency bugs in Node.js,” in *Proceedings of the 32nd IEEE/ACM International Conference on Automated Software Engineering (ASE)*, 2017, pp. 520–531.
- [4] R. Chen, M.-F. Yang, and X.-Y. Guo, “Interrupt data race detection based on shared variable access order pattern,” *Journal of Software*, vol. 27, no. 3, pp. 547–561, 2016.
- [5] M. Yang, B. Gu, Z. Duan, Z. Jin, N. Zhan, and Y. Dong, “Intelligent program synthesis framework and key scientific problems for embedded software,” *Chinese Space Science and Technology*, vol. 42, no. 4, pp. 1–7, 2022.
- [6] A. Chou, J. Yang, B. Chelf, S. Hallem, and D. Engler, “An empirical study of operating systems errors,” in *Proceedings of the 8th ACM Symposium on Operating Systems Principles (SOSP)*, 2001, pp. 73–88.

- [7] N. Palix, G. Thomas, S. Saha, C. Calvès, J. Lawall, and G. Muller, “Faults in Linux: Ten years later,” in *Proceedings of the 16th International Conference on Architectural Support for Programming Languages and Operating Systems (ASPLOS)*, 2011, pp. 305–318.
- [8] Y. Wang, J. Shi, L. Wang, J. Zhao, and X. Li, “Detecting data races in interrupt-driven programs based on static analysis and dynamic simulation,” in *Proceedings of the 7th Asia-Pacific Symposium on Internetware (Internetware)*, 2015, pp. 199–202.
- [9] Y. Wang, L. Wang, T. Yu, J. Zhao, and X. Li, “Automatic detection and validation of race conditions in interrupt-driven embedded software,” in *Proceedings of the 26th ACM SIGSOFT International Symposium on Software Testing and Analysis (ISSTA)*, 2017, pp. 113–124.
- [10] W. Yu, F. Gao, L. Wang, T. Yu, J. Zhao, and X. Li, “Automatic detection, validation and repair of race conditions in interrupt-driven embedded software,” *IEEE Transactions on Software Engineering*, vol. 48, no. 2, pp. 346–363, 2022.
- [11] S. Lu, J. Tucek, F. Qin, and Y. Zhou, “AVIO: Detecting atomicity violations via access interleaving invariants,” *ACM SIGOPS Operating Systems Review*, vol. 40, no. 5, pp. 37–48, 2006.
- [12] H. Fu, Z. Wang, X. Chen, and X. Fan, “A systematic survey on automated concurrency bug detection, exposing, avoidance, and fixing techniques,” *Software Quality Journal*, vol. 26, pp. 855–889, 2018.
- [13] C. Von Praun and T. R. Gross, “Static detection of atomicity violations in object-oriented programs,” *Journal of Object Technology*, vol. 3, no. 6, pp. 103–122, 2004.
- [14] C. Flanagan and S. N. Freund, “Atomizer: A dynamic atomicity checker for multithreaded programs,” *ACM SIGPLAN Notices*, vol. 39, no. 1, pp. 256–267, 2004.
- [15] R. Agarwal, A. Sasturkar, L. Wang, and S. D. Stoller, “Optimized run-time race detection and atomicity checking using partial discovered types,” in *Proceedings of the 20th IEEE/ACM International Conference on Automated Software Engineering (ASE)*, 2005, pp. 233–242.

- [16] L. Wang and S. D. Stoller, “Accurate and efficient runtime detection of atomicity errors in concurrent programs,” in *Proceedings of the 11th ACM SIGPLAN Symposium on Principles and Practice of Parallel Programming (PPoPP)*, 2006, pp. 137–146.
- [17] V. Kahlon, Y. Yang, S. Sankaranarayanan, and A. Gupta, “Fast and accurate static data-race detection for concurrent programs,” in *Proceedings of the 19th International Conference on Computer Aided Verification (CAV)*, 2007, pp. 226–239.
- [18] Z. Letko, T. Vojnar, and B. Krena, “Atomrace: Data race and atomicity violation detector and healer,” in *Proceedings of the 6th workshop on Parallel and distributed systems: testing, analysis, and debugging (PADTAD)*, 2008, pp. 1–10.
- [19] Q. Chen, L. Wang, Z. Yang, and S. D. Stoller, “HAVE: Detecting atomicity violations via integrated dynamic and static analysis,” in *Proceedings of the 12th International Conference on Fundamental Approaches to Software Engineering (FASE)*, 2009, pp. 425–439.
- [20] L. Chew and D. Lie, “Kivati: Fast detection and prevention of atomicity violations,” in *Proceedings of the 5th European conference on Computer systems (EuroSys)*, 2010, pp. 307–320.
- [21] R. J. Dias, V. Pessanha, and J. M. Lourenço, “Precise detection of atomicity violations,” in *Proceedings of the 8th International Haifa Verification Conference on Hardware and Software (HVC)*, 2012, pp. 8–23.
- [22] P. Wang, J. Krinke, X. Zhou, and K. Lu, “Avpredictor: Comprehensive prediction and detection of atomicity violations,” *Concurrency and Computation: Practice and Experience*, vol. 31, no. 15, p. e5160, 2019.
- [23] C. Li, R. Chen, B. Wang, T. Yu, D. Gao, and M. Yang, “Precise and efficient atomicity violation detection for interrupt-driven programs via staged path pruning,” in *Proceedings of the 31st ACM SIGSOFT International Symposium on Software Testing and Analysis (ISSTA)*, 2022, pp. 506–518.
- [24] B. Yu, C. Tian, H. Xing, and Z. Yang, “Detecting atomicity violations in interrupt-driven programs via interruption points selecting and delayed ISR-triggering,” in *Proceedings of the 31st ACM Joint European*

Software Engineering Conference and Symposium on the Foundations of Software Engineering (ESEC/FSE), 2023, pp. 1153–1164.

- [25] Y. Zhang, L. Qu, Y. Wu, L. Wu, T. Yu, R. Chen, and W. Kong, “Bounded verification of atomicity violations for interrupt-driven programs via lazy sequentialization,” *ACM Transactions on Software Engineering and Methodology*, vol. 34, no. 3, pp. 1–33, 2025.
- [26] D. Kroening and M. Tautschnig, “CBMC-C bounded model checker,” in *Proceedings of the 20th International Conference on Tools and Algorithms for the Construction and Analysis of Systems (TACAS)*, 2014, pp. 389–391.
- [27] N. Eén and N. Sörensson, “An extensible SAT-solver,” in *Proceedings of the 6th International Conference on Theory and Applications of Satisfiability Testing (SAT)*, 2003, pp. 502–518.
- [28] J. Regehr, “Random testing of interrupt-driven software,” in *Proceedings of the 5th ACM International Conference on Embedded Software (EMSOFT)*, 2005, pp. 290–298.
- [29] M. Pan, S. Chen, Y. Pei, T. Zhang, and X. Li, “Easy modelling and verification of unpredictable and preemptive interrupt-driven systems,” in *Proceedings of the 41st IEEE/ACM International Conference on Software Engineering (ICSE)*, 2019, pp. 212–222.
- [30] M. Vaziri, F. Tip, and J. Dolby, “Associating synchronization constraints with data in an object-oriented language,” *ACM SIGPLAN Notices*, vol. 41, no. 1, pp. 334–345, 2006.
- [31] H. Feng, L. Yin, W. Lin, X. Zhao, and W. Dong, “Rchecker: A CBMC-based data race detector for interrupt-driven programs,” in *Proceedings of the 20th International Conference on Software Quality, Reliability and Security Companion (QRS-C)*, 2020, pp. 465–471.
- [32] L. Liang, T. Melham, D. Kroening, P. Schrammel, and M. Tautschnig, “Effective verification for low-level software with competing interrupts,” *ACM Transactions on Embedded Computing Systems*, vol. 17, no. 2, pp. 1–26, 2017.

- [33] D. Kroening, L. Liang, T. Melham, P. Schrammel, and M. Tautschnig, “Effective verification of low-level software with nested interrupts,” in *Proceedings of the 2015 Design, Automation & Test in Europe Conference & Exhibition (DATE)*, 2015, pp. 229–234.
- [34] P. Darke, B. Chimdyalwar, R. Venkatesh, U. Shrotri, and R. Metta, “Over-approximating loops to prove properties using bounded model checking,” in *Proceedings of the 2015 Design, Automation & Test in Europe Conference & Exhibition (DATE)*, 2015, pp. 1407–1412.
- [35] B. Chimdyalwar, P. Darke, A. Chavda, S. Vaghani, and A. Chauhan, “Eliminating static analysis false positives using loop abstraction and bounded model checking,” in *Proceedings of the 20th International Symposium on Formal Methods (FM)*, 2015, pp. 573–576.
- [36] X. Ma, I. Ashraf, and W. K. Chan, “Davida: A decentralization approach to localizing transaction sequences for debugging transactional atomicity violations,” *IEEE Transactions on Reliability*, vol. 72, no. 2, pp. 808–826, 2022.
- [37] X. Chang, W. Dou, Y. Gao, J. Wang, J. Wei, and T. Huang, “Detecting atomicity violations for event-driven node.js applications,” in *Proceedings of the 41st International Conference on Software Engineering (ICSE)*, 2019, pp. 631–642.
- [38] C. Flanagan, S. N. Freund, M. Lifshin, and S. Qadeer, “Types for atomicity: Static checking and inference for Java,” *ACM Transactions on Programming Languages and Systems*, vol. 30, no. 4, pp. 1–53, 2008.
- [39] C. Flanagan and S. Qadeer, “A type and effect system for atomicity,” *ACM SIGPLAN Notices*, vol. 38, no. 5, pp. 338–349, 2003.
- [40] A. Sasturkar, R. Agarwal, L. Wang, and S. D. Stoller, “Automated type-based analysis of data races and atomicity,” in *Proceedings of the 10th ACM SIGPLAN Symposium on Principles and Practice of Parallel Programming (PPoPP)*, 2005, pp. 83–94.
- [41] C. Wang, R. Limaye, M. Ganai, and A. Gupta, “Trace-based symbolic analysis for atomicity violations,” in *Proceedings of the 16th International Conference on Tools and Algorithms for the Construction and Analysis of Systems (TACAS)*, 2010, pp. 328–342.

- [42] B. Kasikci, W. Cui, X. Ge, and B. Niu, “Lazy diagnosis of in-production concurrency bugs,” in *Proceedings of the 26th Symposium on Operating Systems Principles (SOSP)*, 2017, pp. 582–598.
- [43] X. Du, L. Yin, H. Feng, and W. Dong, “Program verification enhanced precise analysis of interrupt-driven program vulnerabilities,” in *Proceedings of the 28th Asia-Pacific Software Engineering Conference (APSEC)*, 2021, pp. 253–263.